

Procedural 3D Creature Assembly and Gap-Free Stitching in libGDX

Introduction to Procedural Chimera Morphology

The procedural generation of three-dimensional entities via the arbitrary runtime assembly of pre-authored mesh components presents a complex morphological, geometric, and pipeline-oriented challenge. In the specific context of generating cross-species "chimera" entities from disjointed 3D assets—such as grafting a chicken head or dog leg onto a horse torso—the underlying software architecture must resolve severe discrepancies in scale, coordinate space, and topological alignment. The ultimate objective is to produce a contiguous, gap-free silhouette suitable for off-screen rendering into an orthographic framebuffer, yielding stylized, palette-quantized two-dimensional pixel art across four cardinal directions.

The current stitching implementation suffers from visible anatomical disconnections, floating appendages, and internal mesh collisions. These issues stem from a reliance on naive Axis-Aligned Bounding Box (AABB) spatial logic, improper handling of affine transformations within the libGDX rendering pipeline, and the absence of interstitial geometry required to bridge disparate mesh topologies. This report provides an exhaustive, mathematically rigorous roadmap for solving these structural defects. It establishes a robust framework for procedural scale normalisation, algorithmic extraction of anatomical attachment points directly from vertex data, and the real-time generation of bridging geometry. Furthermore, it addresses the strict constraints of Java and libGDX memory management, single-threaded OpenGL context operations, and depth-buffer integrity during the off-screen baking process.

Coordinate Space Transformations and Bounding Box Mechanics

A foundational requirement for procedural 3D assembly is the accurate mathematical representation of mesh boundaries in world space. The legacy stitching logic extracts geometric extents using `new ModelInstance(model).calculateBoundingBox(tBox)`, followed by manual translation and uniform scaling operations derived from a JSON-defined `unitScale` of 0.01. This approach bypasses the intended transformation hierarchy of the libGDX engine, leading to profound alignment errors.¹

The Local Versus World Space Fallacy

In standard three-dimensional rendering paradigms, a raw mesh exported from a digital content creation tool, such as Blender, exists in model space (frequently referred to as local space).¹ When `calculateBoundingBox()` is invoked on a freshly instantiated `ModelInstance` lacking an explicitly applied transformation matrix, the resulting `BoundingBox` represents the extents of the

raw vertices in the original model space.³

The legacy code attempts to compensate for this mathematical disconnect by extracting the local centre coordinates and subsequently applying a manual translation and scale. While mathematically linear, this operation forces the assembler to constantly manage local-to-world space conversions manually, thereby destroying the engine's ability to maintain a coherent scene graph.

The correct and robust methodology within the libGDX framework dictates that the `ModelInstance` should first receive its spatial transformation—comprising scale, rotation, and translation—and the bounding box should then be multiplied by that transformation matrix to achieve true world-space coordinates.⁴

The formula for obtaining an accurate world-space bounding box is fundamentally a matrix multiplication operation where the local bounding vectors are transformed by the world matrix. In libGDX syntax, this is efficiently executed by chaining the transformation directly to the bounding box instance.⁴ By applying the 0.01 scale factor directly to the instance's transformation matrix before any alignment calculations are executed, all subsequent spatial operations—including gap-bridging and heuristic measurements—can occur in a unified, metric world-space coordinate system. This paradigm shift eliminates the cognitive and computational overhead of tracking which variables represent Blender centimetres and which represent world metres.

Matrix Hierarchy and Affine Transformations

Properly managing the transformation hierarchy is critical when assembling multipart characters. The torso acts as the root node of the biological hierarchy. When the `CreatureBaker` requests a facing rotation to render the North, East, South, or West sprites, rotating the torso must inherently rotate all attached limbs while preserving their relative spatial offsets.

If limbs are simply placed at specific world coordinates without preserving their local offset relative to the torso, applying a global rotation around the Y-axis will tear the creature apart. The libGDX `Matrix4` class handles affine transformations, allowing the system to bake the translation and scale into a base matrix for each limb.⁵ During the facing update, the system must create a master rotation matrix and multiply it by the base matrix of each component. This preserves the internal skeletal structure of the chimera without requiring an explicit bone hierarchy. The current `StitchedCreature.java` implementation correctly caches `torsoBase` and `limbBases`, but these bases must be established using fully resolved world-space coordinates derived from the newly proposed scaling and attachment heuristics.

Cross-Species Scale Normalisation Strategies

The most jarring visual artefacts in cross-species chimera generation arise from volumetric and proportional discrepancies.⁶ Placing a mammalian leg—such as that of a horse, inherently scaled to support a massive volume—underneath an avian torso like a chicken results in an appendage that completely dwarfs the core body. Standardizing these disparate parts requires a

deliberate and mathematically sound scaling strategy.

Dominant-Axis Proportional Scaling

Instead of relying solely on the raw metric dimensions of the imported assets, the system must employ a hierarchical, proportion-enforcing algorithm. This algorithm establishes the TORSO as the primary anatomical anchor. All peripheral parts, including limbs, heads, and tails, must calculate their final scale dynamically based on a fixed ratio relative to the torso's dominant axis, which is typically the Y-axis for bipedal or quadrupedal structures.⁷

The process of data normalisation can be adapted to three-dimensional meshes to enforce this anatomical constraint.⁸ Rather than scaling a part uniformly by its fixed `unitScale`, the assembler must calculate a dynamic multiplier. Let the world-space height of the torso bounding box represent the core metric. Let the raw local-space height of the limb mesh represent the unscaled asset. A target anatomical ratio must be predefined for each socket type. The dynamic scale factor applied to the limb's transform is derived by multiplying the torso height by the target ratio and dividing by the unscaled limb height.

This strategy guarantees that an elephant leg and a chicken leg will both occupy the exact same vertical real estate when attached to any given torso. This normalises the alien anatomy into a cohesive framework, ensuring that the final silhouette fits reliably within the 2.2 by 2.2 metre orthographic camera viewport without protruding off-screen or vanishing into microscopic dimensions.

Socket Type	Target Anatomical Ratio Relative to Torso Height	Rationale for Normalised Proportions
HEAD	40% to 50%	Ensures the head remains identifiable in 256x256 pixel art without occluding the torso silhouette.
LEG_L / LEG_R	60% to 75%	Provides a stable centre of gravity. Extreme ratios (e.g., >100%) push the torso out of the camera viewport.
ARM_L / ARM_R	50% to 65%	Maintains distinct silhouettes for side-profiles (East/West

		renders) preventing arms from blending into legs.
TAIL	30% to 40%	Keeps the tail as a secondary detail rather than a dominant visual mass.

The Fallacy of Volumetric Scaling

Alternative theoretical approaches might suggest scaling based on bounding box volume, operating under the assumption that making the volume of a chicken leg equal to the volume of a horse leg would yield visual parity. However, volumetric scaling causes catastrophic deformation when aspect ratios differ wildly across species.¹⁰

A long, thin appendage scaled to match the volume of a short, stout appendage will become impractically gigantic along its longitudinal axis to compensate for its lack of girth. Conversely, scaling a thick appendage to match the volume of a thin one will shrink it to a negligible height. Therefore, referencing a single, dominant one-dimensional axis—specifically height or length—is the only mathematically stable approach for organic procedural generation.

By enforcing uniform axis scaling based exclusively on the height ratio, the algorithm preserves the original aspect ratio of the mesh (e.g., a horse leg remains thick and muscular, while a flamingo leg remains thin and spindly), but ensures they both reach from the torso socket to the floor plane.

Algorithmic Attachment-Point Heuristics

The legacy codebase operates under the assumption that the absolute highest coordinate of a limb's bounding box represents its anatomical attachment joint, and the absolute lowest coordinate of the torso represents its respective socket. This assumption fails severely when applied to organic three-dimensional models.¹¹

A bounding box is an axis-aligned prism¹²; it encompasses the widest spatial extremes of the entire mesh. If a canine model features a raised tail or perked ears, the maximum Y-value of the torso bounding box might rest at the tip of the tail or ear, far removed from the anatomical shoulder or neck joint. Snapping a head to the height of a raised tail results in extreme floating geometry. To achieve gap-free stitching without requiring technical artists to manually author socket metadata via vertex colouring or embedded skeletal bone data¹³, the software must execute a heuristic analysis of the raw mesh geometry at runtime.

Extremal Vertex Slice Centroid Extraction

The most accurate method to deduce an anatomical joint from an unrigged, static mesh without external metadata is to iterate over its raw vertex data and calculate the spatial centroid of the vertices located exclusively at the anatomical extremes.

In the libGDX framework, a Mesh stores its vertex data within a flat, one-dimensional floating-point array.¹⁴ Because this array interleaves positions, normal vectors, and texture coordinates based on the VertexAttributes memory layout, parsing it requires precise byte stride and offset calculations.¹⁵

To locate a top-attachment point—such as the top of a severed leg intended for attachment to a torso—the algorithm must first retrieve the floating-point array from the mesh. It must then calculate the vertex size in floats and find the specific memory offset for the position attribute.¹⁷ By iterating through the array, the algorithm records the absolute maximum and minimum coordinates along the dominant axis in local space.

Once the total height of the mesh is established, the algorithm defines an extraction slice. For a top joint, it isolates all vertices located within the highest five to ten percent of the mesh geometry.¹⁸ The algorithm then calculates the average X, Y, and Z coordinates of only the vertices contained within this narrow slice.

Algorithmic Step	Mathematical / API Operation	Purpose within the Heuristic Pipeline
1. Memory Mapping	<pre>int stride = mesh.getVertexSize() / 4; int posOffset = attr.offset / 4;</pre>	Allows safe traversal of the interleaved 1D float array to extract solely spatial XYZ data without reading UVs or normals.
2. Extent Bounding	$Y_{max} = \max(Y_0 \dots Y_n)$ $Y_{min} = \min(Y_0 \dots Y_n)$	Determines the absolute local height of the unscaled part to calculate the percentage threshold for slicing.
3. Slice Thresholding	$Y_{thresh} = Y_{max} - 0.10 \times ($	Isolates the anatomical "cut-off" geometry (e.g., the severed neck of a head model) while

		ignoring the bulk mass below.
4. Centroid Accumulation	$C_{xyz} = \frac{1}{k} \sum_{i=1}^k V_{i(xyz)}$	Averages the coordinates of the k vertices within the slice to find the true centre of mass of the attachment surface.

This computed coordinate, termed the Extremal Centroid, accurately represents the centre of mass of the mesh's physical biological cut-off point. This approach completely ignores peripheral geometric noise, such as tails, horns, or bent toes, that would otherwise corrupt a standard bounding box calculation. By applying this algorithm to the bottom of the torso and the top of the legs, the assembler obtains highly precise, mathematically derived biological sockets.

Intentional Penetration and Intersection Mechanics

Once the limb centroid and torso socket centroid are independently calculated and transformed into world space, naive point-to-point snapping will still leave a visible seam. This occurs due to the inherent curvature and topological irregularity of spherical organic meshes. To eliminate gaps entirely, a calculated penetration offset must be applied.

Rather than placing the limb centroid exactly at the torso socket centroid, the limb must be translated inward along the torso's geometric normal vector. In a simplistic vertical assembly, the limb is forced upwards along the Y-axis into the torso by a factor of targeted overlap. The final penetration vector is defined by translating the socket position inward by a factor approximately equal to ten to fifteen percent of the limb's total scaled height.

While this deliberate offset causes internal geometric clipping and self-intersection within the mesh data, the depth-buffered off-screen render resolves the external surface perfectly.¹⁹ Because the final output is a two-dimensional stylised pixel-art sprite, the internal clipping is fundamentally invisible to the player, while the external silhouette becomes perfectly contiguous.

Resolving Topological Gaps via Procedural Interstitial Geometry

Even with rigorous proportional scaling and centroid-based penetration, severe morphological divergence can result in aesthetic failures. For instance, stitching the microscopic, spindly neck of a chicken to the massive, broad torso of a horse can result in a "shelf" effect, where the silhouette pinches unnaturally and exposes the flat geometric cut of the torso. To guarantee a gap-free, contiguous solid under all randomised permutations, the runtime system must

programmatically generate bridging geometry to act as organic connective tissue.²⁰

Leveraging the ModelBuilder and MeshPartBuilder API

The libGDX architecture provides the ModelBuilder and MeshPartBuilder application programming interfaces to construct three-dimensional geometry dynamically at runtime.²² These tools can instantiate geometric primitives—such as cylinders, spheres, and capsules—using pure Java code without reliance on external asset loading.²⁴ By generating a physical bridge between the torso socket centroid and the limb attachment centroid, the system forces an absolute topological continuity.

Creating a procedural cylinder between two arbitrary points in three-dimensional space requires specific vector mathematics to orient, stretch, and scale the primitive accurately.²⁵ Let the torso socket centroid be Point A, and the limb attachment centroid be Point B. The required height of the cylinder is the Euclidean distance between A and B. The spatial position of the cylinder must be the midpoint between A and B. The orientation of the cylinder is defined by a direction vector pointing from A to B.

In libGDX, a cylinder generated via `MeshPartBuilder.cylinder()` defaults to resting vertically along the Y-axis.²⁶ To align the cylinder so it bridges the two centroids, a `Matrix4` must be constructed using a quaternion rotation. This rotation maps the default Y-axis to point towards the calculated direction vector. The transformation matrix is generated by scaling the cylinder to the required distance, translating it to the midpoint, and applying the quaternion rotation.

By applying this transformation matrix via the `meshBuilder.setVertexTransform(transform)` method prior to invoking the `meshBuilder.cylinder(...)` command, the primitive geometry is explicitly drawn in the exact spatial void between the two biological segments.¹⁴

Ball-and-Socket Masking for Tapered Joints

Standard cylinders possess uniform top and bottom radii. When bridging a massive torso socket to a tiny limb attachment point, a uniform cylinder creates a harsh, mechanical step that destroys the organic illusion.²¹ A tapered shape, mathematically defined as a truncated cone, is anatomically superior for connecting differently sized biological masses.

While the MeshPartBuilder provides a `cone()` method, orienting a partial cone to act as a seamless tapered cylinder is highly complex.²² Given the strict constraint of executing the assembly within a single runtime Java class while maintaining optimal performance, a highly effective and pragmatic heuristic is the Ball-and-Socket Masking technique.

Instead of calculating complex tapered mathematics, the assembler utilizes the MeshPartBuilder to place a `sphere()` directly at the torso socket, scaled to match the torso's local geometric width. A second sphere() is generated at the limb attachment point, scaled to match the much smaller width of the limb.²² A standard, thin cylinder is then drawn between the centres of these two spheres to act as the core skeletal bridge. The spheres effectively mask the harsh orthogonal edges of the cylinder, creating a smooth, continuous, albeit alien, organic joint that smoothly

transitions from the large mass to the small mass.

All primitive bridging geometry must be assigned the exact same Material and shader attributes as the base .g3db meshes. This ensures that the downstream creature_stylize.frag shader calculates the lighting gradients and Sobel edge detections uniformly across the entire entity, blending the procedural tissue seamlessly into the pre-authored assets.

Advanced Orthographic Rendering and Framebuffer Pipeline

The architectural requirement to bake these dynamically generated, high-poly chimeras into a tiny 256 by 256 pixel orthographic framebuffer introduces distinct constraints on how the constructed models are handled in Graphics Processing Unit (GPU) memory and how the rendering pipeline processes depth.

Depth Buffer Precision and Z-Fighting Mitigation

The off-screen pipeline utilizes an OrthographicCamera set to a 2.2 by 2.2 metre viewport, looking from (0, 0.8, 5) to (0, 0.8, 0). The depth buffer is correctly enabled on the sceneFbo object via the hasDepth = true flag.

A critical vulnerability in procedural generation is the interaction between the depth buffer and overlapping geometry. When limbs are forced to penetrate the torso, and procedural spheres and cylinders are packed into the intersection points to eliminate gaps, the overlapping coplanar faces cause intense z-fighting if the depth buffer lacks sufficient precision.

The near and far planes of the orthographic bake camera dictate the resolution of the depth buffer. Setting the planes too wide (e.g., 0.1f to 1000f) spreads the 24-bit depth precision over a massive distance, leading to z-fighting among closely packed vertices. The current pipeline sets the near plane to 0.1f and the far plane to 20f. Given that the camera is positioned at Z=5 and looks at Z=0, the creature geometry occupies the space between Z=4 and Z=6. This extremely tight frustum utilizes the depth buffer's precision optimally, virtually eliminating rendering artefacts on overlapping procedural geometry.

Stylisation Pass and Alpha Edge Detection

The final stage of the rendering pipeline is the stylisation pass (creature_stylize.frag), which relies on a 3x3 Sobel kernel applied to the alpha channel to render hard, thick pixel-art outlines around the chimera.

For the Sobel filter to perceive the stitched chimera as a single, contiguous object rather than a collection of overlapping parts with internal lines, it is absolutely imperative that the 3D scene renders against a completely transparent background. The invocation of Gdx.gl.glClearColor(0f, 0f, 0f, 0f) combined with GL20.GL_COLOR_BUFFER_BIT ensures the framebuffer is cleared to zero alpha.

Because the internal overlapping meshes (the penetrating limbs and procedural ball-and-socket

joints) all render their opaque pixels into the RGB channels with an alpha of 1.0, the internal seams possess no alpha gradients whatsoever. Consequently, when the fragment shader executes the Sobel edge detection algorithm exclusively on the alpha channel float `edge = alphaEdge(v_uv);`, it successfully traces only the absolute outer contiguous silhouette of the combined meshes and the procedural bridging geometry. The internal structural chaos is ignored entirely, resulting in a flawless, clean pixel-art aesthetic that perfectly mimics a unified 2D sprite.

Pragmatic Java Implementation Strategy and Optimisation

Creating new Model instances via ModelBuilder for every single bridging primitive generated at runtime will quickly lead to native memory leaks if the Java Garbage Collector is not strictly managed. Each call to ModelBuilder.end() instantiates a Mesh whose vertex data is stored in unmanaged Video RAM (VRAM).²³ When a random creature is defeated or discarded by the game logic, the Model and its associated procedural meshes must be explicitly disposed of by invoking model.dispose(). Relying on Java's automated garbage collection will not free the OpenGL VRAM, leading to a rapid OutOfMemory crash.

Draw Call Optimisation via ModelCache

A fully assembled procedural chimera consists of one torso, five limbs, and potentially ten to fifteen procedural bridging primitives (spheres and cylinders). Rendering these as individual ModelInstance objects within a ModelBatch generates between sixteen and twenty-one separate draw calls per frame.²⁸ While this is technically acceptable for offline, synchronous baking where frame rate is not a hard constraint, it is highly sub-optimal and impacts generation speed.

The most efficient libGDX solution for this architectural bottleneck is to utilise the ModelCache class.²⁹ The ModelCache operates analogously to a SpriteBatch, but is designed specifically for static 3D geometry.

By beginning the cache, iterating through all the individual ModelInstance limbs, torsos, and procedural connectors, and finally ending the cache, libGDX merges all disparate meshes that share identical materials into a single, unified static mesh residing in VRAM.

Standard Rendering (ModelBatch)	Cached Rendering (ModelCache)	Optimisation Impact
Issues 16-21 separate GL draw calls per angle bake.	Issues 1 to 2 GL draw calls per angle bake.	Reduces CPU-to-GPU command overhead by ~90%.

Requires iterating through a HashMap of limbs during render.	Renders a single contiguous block of geometry.	Minimises Java loop overhead in the rendering thread.
Global rotation requires applying a Matrix4 multiplication to 20 individual objects sequentially.	Global rotation requires applying a single Matrix4 multiplication to the cache.	Vastly simplifies the setFacing() logic for N/E/S/W generation.

When the CreatureBaker calls `creature.setFacing(yDegrees)` to capture the North, East, South, and West sprites, rotating the camera around the single ModelCache—or applying a single rotational transform to the cache itself—drastically reduces matrix multiplication overhead during the multi-angle capture process.

The Unified Assembly Algorithm

To satisfy all constraints and successfully transition from the flawed, disconnected logic to a robust, gap-free procedural assembly within a single Java class, the system must execute the following sequential pipeline:

First, the system retrieves the randomly selected torso model and applies the predefined `unitScale` directly to its local transformation matrix. It then computes the true world-space bounding box to determine the dominant height metric of the creature's core.

Subsequently, the algorithm iterates through each required anatomical socket. It retrieves the appropriate limb asset, calculates its unscaled local height, and derives a dynamic world scale based on the target proportional fraction (e.g., ensuring a leg is precisely 60% of the torso's height). This dynamic scale is applied to the limb's matrix.

With both torso and limb accurately scaled in world space, the algorithm executes the Extremal Vertex Slice technique. It parses the floating-point vertex arrays of the meshes, isolating the top ten percent of the limb geometry and the bottom ten percent of the torso socket geometry. By calculating the centroids of these vertex clusters, the algorithm identifies the precise biological attachment vectors.

The limb is then translated in world space so that its attachment vector aligns with the torso's socket vector. To guarantee intersection, a penetration scalar is applied, forcing the limb inward along the geometric normal.

Following translation, the procedural connective tissue phase begins. The ModelBuilder API is invoked to generate perfectly scaled spherical meshes at both the socket and attachment centroids, effectively creating an organic ball-and-socket joint. A procedural cylinder is then

extruded between these spheres, masking any remaining geometric voids.

Finally, all pre-authored models and procedurally generated primitives are baked into a libGDX ModelCache. This cache is passed to the CreatureBaker, where the orthographic camera and Sobel-edge shader flatten the highly complex, mathematically bonded three-dimensional chimera into perfectly contiguous, chunk-quantised, gap-free two-dimensional sprite sheets.

Conclusion

The pursuit of gap-free, procedural three-dimensional creature assembly across wildly divergent species morphologies cannot be solved through the linear spatial translation of static bounding boxes. The inherent irregularity of organic shapes demands a holistic, geometrically aware algorithm. By shifting from local coordinate assumptions to absolute world-space matrix transformations, a mathematically stable foundation is established. Substituting raw, arbitrary dimensional scaling with Proportional Torso-Relative Scaling ensures that all biological appendages, regardless of their origin, exist within a unified and predictable aesthetic volume.

Furthermore, replacing naive bounding box limits with Extremal Vertex Slice Centroid Extraction empowers the software to autonomously deduce biological joints from static, unrigged meshes without manual authoring. Executing deliberate geometric penetration paired with dynamically built, ball-and-socket primitive masking guarantees an impenetrable silhouette under all randomised permutations. Ultimately, this comprehensive algorithmic pipeline ensures that the resulting three-dimensional rendering—and its subsequent palette-quantized two-dimensional stylisation—reads to the human eye as a cohesive, seamless, and entirely unified biological entity.

Works cited

1. Coordinate systems - libGDX, accessed May 10, 2026, <https://libgdx.com/wiki/articles/coordinate-systems>
2. ELI5 different spaces/coordinate systems (model space, world space, screen space, NDC, raster space)? : r/GraphicsProgramming - Reddit, accessed May 10, 2026, https://www.reddit.com/r/GraphicsProgramming/comments/1jcoww0/eli5_different_spacescoordinate_systems_model/
3. Models - libGDX, accessed May 10, 2026, <https://libgdx.com/wiki/graphics/3d/models>
4. How to get bounding box for model instances in libgdx? - Stack Overflow, accessed May 10, 2026, <https://stackoverflow.com/questions/20932527/how-to-get-bounding-box-for-model-instances-in-libgdx>
5. 3D transform system similar to Unity3D for libGDX - GitHub Gist, accessed May 10, 2026, <https://gist.github.com/prodigga/e0ceb4d9e91ca8e0cf8e>
6. Procedural Creature Generation and Animation for Games - Bournemouth University, accessed May 10, 2026,

- <https://nccastaff.bournemouth.ac.uk/jmacey/MastersProject/MSc22/01/ProceduralCreatureGenerationandAnimationforGames.pdf>
7. Creature grammar for creative modeling of 3D monsters - Kai Xu, accessed May 10, 2026, https://kevinkaixu.net/papers/guo_gmp14_creature.pdf
 8. Mastering Feature Scaling and Normalization in Machine Learning | by Juan C Olamendy, accessed May 10, 2026, <https://medium.com/@juanc.olamendy/mastering-feature-scaling-and-normalization-in-machine-learning-985fdecfab5c>
 9. Feature scaling - Wikipedia, accessed May 10, 2026, https://en.wikipedia.org/wiki/Feature_scaling
 10. The Bounding Mesh Algorithm - mediaTUM, accessed May 10, 2026, <https://mediatum.ub.tum.de/attfile/1255722/incoming/2015-Jun/140911.pdf>
 11. Finding locations of Vertices on a Mesh - Questions & Answers - Unity Discussions, accessed May 10, 2026, <https://discussions.unity.com/t/finding-locations-of-vertices-on-a-mesh/48607>
 12. Circles, planes, rays, etc. - libGDX, accessed May 10, 2026, <https://libgdx.com/wiki/math-utils/circles-planes-rays-etc>
 13. Getting position of skinned mesh surface? - Unity Discussions, accessed May 10, 2026, <https://discussions.unity.com/t/getting-position-of-skinned-mesh-surface/620157>
 14. Meshes - libGDX, accessed May 10, 2026, <https://libgdx.com/wiki/graphics/opengl-utils/meshes>
 15. LibGDX Meshes Lesson 1 · mattdesl/lwjgl-basics Wiki - GitHub, accessed May 10, 2026, <https://github.com/mattdesl/lwjgl-basics/wiki/LibGDX-Meshes-Lesson-1>
 16. 2D Animation - libGDX, accessed May 10, 2026, <https://libgdx.com/wiki/graphics/2d/2d-animation>
 17. What does LibGDX's mesh.getVerticies output? - Stack Overflow, accessed May 10, 2026, <https://stackoverflow.com/questions/38731806/what-does-libgdxs-mesh-getverticies-output>
 18. 3D mesh direction detection - Stack Overflow, accessed May 10, 2026, <https://stackoverflow.com/questions/61016310/3d-mesh-direction-detection>
 19. New Method for Removing Self-Intersecting 3D Mesh Data - 3DPrint.com, accessed May 10, 2026, <https://3dprint.com/261812/removing-self-intersection-3d-mesh-data-researchers-create-robust-algorithm/>
 20. Procedural Generation of Articulated Simulation-Ready Assets - arXiv, accessed May 10, 2026, <https://arxiv.org/html/2505.10755v3>
 21. Bridging the Gap: Automated Steady Scaffoldings for 3D Printing, accessed May 10, 2026, <https://members.loria.fr/JDumas/files/papers/scaffoldings/bridging-the-gap.pdf>
 22. ModelBuilder (libGDX Core 1.10.0 API) - javadoc.io, accessed May 10, 2026, <https://javadoc.io/static/com.badlogicgames.gdx/gdx/1.10.0/index.html?com/badlogic/gdx/graphics/g3d/utils/ModelBuilder.html>
 23. ModelBuilder, MeshBuilder and MeshPartBuilder - libGDX, accessed May 10,

- 2026,
<https://libgdx.com/wiki/graphics/3d/modelbuilder-meshbuilder-and-meshpartbuilder>
24. libgdx/gdx/src/com/badlogic/gdx/graphics/g3d/Utils/ModelBuilder.java at master - GitHub, accessed May 10, 2026,
<https://github.com/libgdx/libgdx/blob/master/gdx/src/com/badlogic/gdx/graphics/g3d/Utils/ModelBuilder.java>
 25. What's the most efficient way to draw cylinder mesh between two movable points? : r/Unity3D - Reddit, accessed May 10, 2026,
https://www.reddit.com/r/Unity3D/comments/14m5rsz/whats_the_most_efficient_way_to_draw_cylinder/
 26. How to draw a line between two points in LibGDX, in 3D - Stack Overflow, accessed May 10, 2026,
<https://stackoverflow.com/questions/38928229/how-to-draw-a-line-between-two-points-in-libgdx-in-3d>
 27. Deprecated List (libGDX Core 1.9.12 API) - javadoc.io, accessed May 10, 2026,
<https://javadoc.io/doc/com.badlogicgames.gdx/gdx/1.9.12/deprecated-list.html>
 28. ModelBatch - libGDX, accessed May 10, 2026,
<https://libgdx.com/wiki/graphics/3d/modelbatch>
 29. ModelCache - libGDX, accessed May 10, 2026,
<https://libgdx.com/wiki/graphics/3d/modelcache>
 30. Combining Model Instances in Libgdx - java - Stack Overflow, accessed May 10, 2026,
<https://stackoverflow.com/questions/35873922/combining-model-instances-in-libgdx>